

Database Coursework Part 2

HLUM HTET AUNG

40841063

Table of Contents

Feedback Updates	5
Union Table Separation.....	5
Additional Attributes for technicians and traffic controllers	5
Updated Justifications/Assumptions (From Part 1)	6
Justification/Assumption for keys.....	7
Updated Queries and Test Plan	7
Aircraft Test Performance Summary (Query 1).....	7
Technician Annual Performance (Query 2)	7
Recently Examined Traffic Controllers (Query 3).....	8
Aircraft safety check (Query 4)	8
Technician Expertise and Aircraft model summary (Query 5)	8
Query Test Plane	9
Constraint and Key Test Plan.....	10
Foreign Key Testing.....	10
Primary Key and Length Testing	10
Task C	11
Relational Schema	11
Relational Schema Table.....	12
Normalization	13
First Normal Form (1NF).....	13
Second Normal Form (2NF)	14
Third Normal Form (3NF)	14
Task D	15
DDL and DML statements	15

Data Definition Language (DDL) Implementation.....	15
Union Table.....	15
Employee Table.....	16
Airplane models	17
Airplane Table.....	18
Technicians Table.....	19
Traffic controller table	20
Technician expertise.....	21
Airworthiness tests model table	22
Test events table	23
Data Manipulation Language (DML) Implementation.....	24
Union Table Data Insertion	24
Employee Table Data Insertion.....	25
Airplane Models.....	26
Airplanes	27
Technicians	28
Traffic controllers.....	29
Technician expertise.....	30
Airworthiness Tests.....	31
Test events.....	32
Task E – Queries	34
Query 1 – Aircraft Test Performance Summary.....	34
Query 2 – Employee Annual Performance	35
Query 3 – Recently Examined Traffic Controllers	36
Query 4 – Aircraft safety check	37

Query 5 – Technician Expertise summary	38
Presentation of Query Results.....	39
Query 1.....	39
Query 2.....	39
Query 3.....	40
Query 4.....	41
Query 5.....	42
Constraint Testing	43
Name Length Test	43
Referential Constraints.....	44
Constraint Summary	44

Feedback Updates

This section mainly focuses on updating the database design and test plan so that it matches the feedback.

Union Table Separation

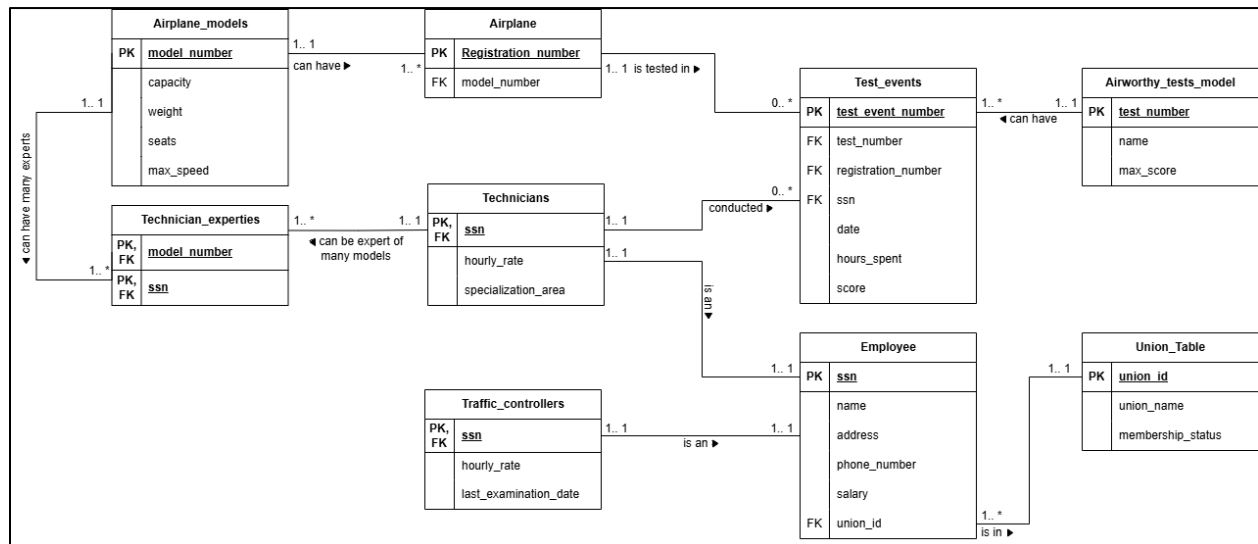


Figure 1 Union Table separated

The union table is separated from employee table for satisfying 3NF in normalization stages. The details for this separation will be discussed further in Normalization (Third Normal Form (3NF)) topic.

Additional Attributes for technicians and traffic controllers

Additional attributes for technicians and controllers are added to ensure that the table is realistic. For technicians, hourly rate and specialization area is added, whereas traffic controllers table has additional hourly rate and last examination date (already exists). These are the unique attributes where Employee table can't summarize. **Hourly rates can be seen in both technicians and traffic controllers.**

Note: The hourly rate in subtype tables, although common, is put on respective tables and not employee table is to avoid null values. In employee table, every employee already has a salary amount but not everyone can have an hourly rate where they are paid for overtime or if an employee is non-hourly employee, who has a set amount of salary.

Updated Justifications/Assumptions (From Part 1)

This section of the report will explain the relations between tables in the airport database. The relation between the tables is –

- (Union Table 1:1-----1: * Employee) – A one to many relationships where an employee can belong to at most one labor union, while a single union can represent multiple employees. Union Table's primary key union_id acts as foreign key in Employee table.
- (Airplane models 1:1-----1: * Airplane) – one to many relations where an airplane model can have many physical registered aircraft, but each plane belongs to one model. Model wide specifications (seats, max speed, capacity, weight) are stored in airplane model tables.
- (Airplane 1:1-----0: * Test events) – one (optional) to many relations where an airplane can optionally have zero or more test events. However, each testing event must be associated with one airplane.
- (Test events 1: *-----1:1 Airworthy test model) – one to many relations where a airworthiness test template can be used across many test events. However, each test event is used in exactly one test model.
- (Test events 0: *-----1:1 Technicians) – optional one to many relations where a test event is conducted by one certified technician, while a technician can perform many test events.
- (Technicians 1:1-----1:1 Employee) – one to one parent and child table relation. Employee stores shared personal attributes for both technicians and traffic controllers, while technician specific attributes are stored in Technicians using ssn as a primary key and foreign key.
- (Traffic controllers 1:1-----1:1 Employee) – table relation and structure are same as technicians and employee, except the specific attributes difference.
- (Technicians 1:1-----1: * Technician expertise) – many to many relations where a technician can be certified for many airplanes
- (Airplane models 1:1-----1: * Technician expertise) – many to many relations, where an airplane models can have one or many certified technicians

There is an original relation between Technicians and Airplane models. However, a composition table is used because it is not possible to store many too many relation without violating first normal form, where there are multi-valued attributes in a table.

Justification/Assumption for keys

This section will discuss the reason for choosing primary keys in the table –

- Natural Primary Key – ssn from Employee table, registration number from Airplane table are chosen because they are guaranteed to be unique for each employee and airplane.
- Surrogate Primary Key – test even number from Test events table, union id from Union table is chosen as auto-incremented surrogate integer keys to avoid using composite keys.
- Shared Primary/Foreign Keys – Technicians table's ssn and Traffic controller's table's ssn are used as both primary key and foreign key to reference employee table. This is to ensure that the relation enforces strict one-to-one relations.

Updated Queries and Test Plan

Aircraft Test Performance Summary (Query 1)

This query monitors how aircraft are tested and how many hours the test event takes, and average test scores the test achieved. This ensures the aircraft are airworthy and can perform flights. By looking at this query management can also monitor the planes' health and performance.

Tables used – Test events, Airplane, Airplane models, Technicians

Aggregated Functions – COUNT, SUM, and AVG

Technician Annual Performance (Query 2)

For this query, management wants to identify which technicians conduct the most aircraft tests each year. This can help management recognize technicians with high performance and allocate testing work, as well as awarding properly.

Tables used – Employee, Technicians, Test events

Aggregated Functions – COUNT, GROUP BY

Recently Examined Traffic Controllers (Query 3)

This query can provide management with the list of traffic controllers who have recently completed their medical examination. With this query, they can determine which controllers have current medical records and who need another examination.

Tables used – Employee, Traffic controllers

Aircraft safety check (Query 4)

This query can help management determine whether the latest test performed on an aircraft has achieved a score that could satisfy historical average score. This can help management determine if a plane is in good condition and if higher than average score the plane is safe to fly.

Tables used – Test events, Airplane, Airplane models, Technicians, Employee

Aggregated Functions – AVG

Technician Expertise and Aircraft model summary (Query 5)

Management wants to identify which technicians are qualified to work on each aircraft model. This is useful for management to assign technicians properly to each plane that require testing or repair.

Tables used – Technician expertise, Technicians, Airplane models, Employee

Query Test Plane

Test Case	Query	Main Test	Expected Result
TC1	Aircraft Test Performance Summary	COUNT, SUM, AVG	Correct test counts, total hours and average scores
TC2	Technician Annual Performance	COUNT, SUM, GROUP BY	Correct tests and total hours per technician
TC3	Recently Examined Traffic Controllers	Date filtering	Only recently examined controller returned
TC4	Aircraft Safety Check	Correlated subquery + AVG	Correct model average and Pass / Review Required results
TC5	Technician Expertise Summary	Multiple joins	Correct technician-model relationships

Table 1 Query Test Plan Table

Expected Result for Query 1 with sample data

Model Number	Registration Number	Technician SSN	Number of Tests	Total Test Hours	Average Test Score
A320	101	1001	3	7.5	88.33
A320	102	1002	2	7.0	75.00
B737	103	1003	2	4.5	90.00

Table 2 Expected Results for Query 1

Expected Result for Query 2 with sample data

SSN	Technician Name	Test Year	Total Tests	Total Hours
1001	John Smith	2026	5	12.5
1002	Sarah Brown	2026	2	7.0

Table 3 Expected Results for Query 2

Expected Result for Query 3 with sample data

SSN	Name	Last Examination Date
1004	Emily Davis	2026-06-15

Table 4 Expected Results for Query 3

Expected Result for Query 4 with sample data

Model Number	Registration Number	Technician Name	Latest Score	Model Average Score	Safety Status
A320	101	John Smith	95	83.00	Pass
A320	102	Sarah Brown	80	83.00	Review Required

Table 5 Expected Results for Query 4

Expected Result for Query 5 with sample data

SSN	Technician Name	Model Number	Specialization Area
1001	John Smith	A320	Maintenance
1001	John Smith	A330	Technical
1002	Sarah Brown	A320	Storage

Table 6 Expected Results for Query 5

Constraint and Key Test Plan

This is the additional test plan that will test the constraints in the database attributes and keys for the tables in the database.

Foreign Key Testing

This test will verify if the foreign key and primary key relation is working properly. A table with the foreign key referencing the primary key of another table will attempt to reference an attribute that doesn't exist. *Sample test, a test event will attempt to reference invalid airworthiness test model key, which will throw a referencing error in MariaDB.*

Primary Key and Length Testing

A simple test where the same primary key will be inserted into the table and the MariaDB will throw a duplicated error, since primary keys can't have duplicates. Same for length constraint test, a sample row with name length violation will be inserted where the database will throw an error.

Task C

Relational Schema

The entity relation diagram of the airport database system includes 3 main tables – Employee, Airplane, and Test Events. These tables have sub-tables, lookup tables, and composite tables that join them together. The relational schema of the database system is –

Employee (social security number, name, address, phone number, salary, *union id*)

Union (union id, union name, membership status)

Airplane (registration number, *model number*)

- *Model number* → **Airplane models** (model number)

Test Events (test event number, *test number*, *registration number*, *social security number*, date, hours spent, score)

- *Test number* → **Airworthy test model** (test number)
- *Registration number* → **Airplane** (Registration number)
- *Social security number* → **Technicians** (social security number)

Technicians (social security number, hourly rate, specialization area)

- *Social security number* → **Employee** (social security number)

Traffic controllers (social security number, hourly rate, last examination date)

- *Social security number* → **Employee** (social security number)

Airworthy test model (test number, name, max score)

Airplane models (model number, capacity, weight, seats, max speed)

Technicians' expertise (model number, social security number)

- *Model number* → **Airplane models** (model number)
- *Social security number* → **Technicians** (social security number)

*Note: The underlined attributes are primary key, and the italic attributes are foreign key.

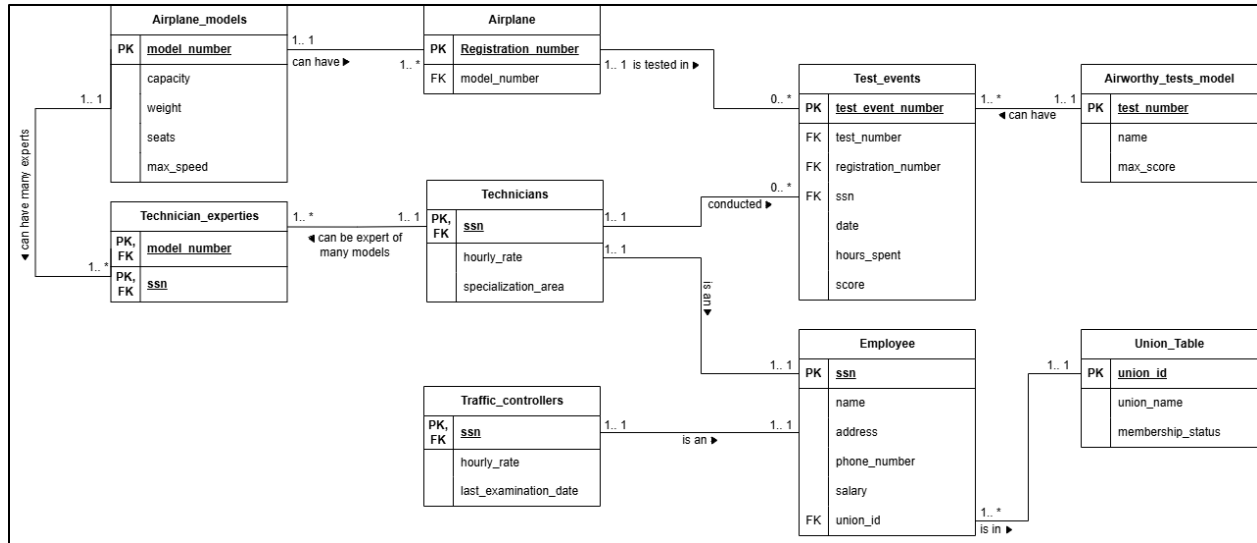


Figure 2 Updated Airport Database ERD

Relational Schema Table

Table Name	Attributes	Primary Key	Foreign Key
Union	Union id, union name, membership status	Union id	None
Employee	Social security number, name, address, phone number, salary, union id	Social security number	Union id
Technicians	Social security number, hourly rate, specialization area	Social security number	Social security number
Traffic controllers	Social security number, hourly rate, last examination date	Social security number	Social security number
Airplane models	Model number, capacity, weight, seats, max speed	Model number	None

Airplane	Registration number, model number	Registration number	Model number
Airworthy test model	Test number, name, max score	Test number	None
Test event	Test event number, test number, registration number, social security number, date, hours spent, score	Test event number	Test number, registration number, social security number
Technicians' expertise	Model number, social security number	Model number, social security number	Model number, social security number

Table 7 Relational Schema Table

Normalization

The database tables are normalized into 3NF. The normalization was done since the implementation of entity relation diagram, with minor changes in part 2.

First Normal Form (1NF)

The tables in the database are in first normal form because –

- Singular values – All attributes store single value in each cell. For example, phone number column stores a single string, score stores a single numerical value.
- No Repeating Groups – The database uses Technicians' expertise composite table to capture expertise relationships as individual rows, instead of storing multiple certifications inside Technicians table.
- Primary Keys – Every relation in the schema has a defined Primary Key.

Second Normal Form (2NF)

As the table already satisfy first normalization, it also satisfies second normal form as the attributes have –

- **Single-Column Primary Key** – For every table except for bridge tables, the primary key consists of a single attribute. For this reason, every attribute in the table depends solely on their primary key and partial dependency is impossible.
- **Composite Primary Key** – For the composite tables the composite keys model number and social security number are the only attributes and there are no non-key attributes in the table. Thus, there is no partial key dependency in this table.

Third Normal Form (3NF)

The tables from the database are also in 3NF because –

- They satisfy both 1NF and 2NF.
- **Separation of Union Data** – After separating union table and the employee table, the transitive attributes in the employee tables, where a non-key attribute depending on another non-key attribute is eliminated.
- **Separation of Model Specifications** – Storing model specifications (capacity, weight, seats, max speed) directly in Airplane table like in part 1 diagram creates transitive dependency. To fix this, these dependencies are relocated into Airplane Model table to ensure non-key attributes in Airplane table depend directly on its primary key (registration number).
- **Subtype Tables (Technicians and Traffic controllers)** – While attributes that are common for both technicians and traffic controllers are stored in Employee table, they also have their specific values (specialization area, hourly rate, and last examination dates) that depend directly on their primary key social security number within Technicians and Traffic controllers' tables.

Task D

DDL and DML statements

This section includes the DDL and DML statements used for table creation and data insertion into those tables. There are total of 9 tables and there are minimum of 15 rows in each table, with employee table being the only table with 30 rows (15 Technicians and 15 Traffic controllers). For DDL statements, the statements will be the screenshot captures of MariaDB executions, and DML statements will be the screenshot of notepad. The statements are tested in the local machine first to check constraints and keys to the design.

Data Definition Language (DDL) Implementation

This section includes implementation for table creation. The tables in the database are created as the entity relation diagram illustrate with the same constraints and relations.

Union Table

```
MariaDB [40841063]> CREATE TABLE Union_Table (
    -> union_id INT AUTO_INCREMENT PRIMARY KEY,
    -> union_name VARCHAR(20) NOT NULL,
    -> membership_status VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.006 sec)

MariaDB [40841063]> show tables;
+-----+
| Tables_in_40841063 |
+-----+
| Union_Table        |
+-----+
1 row in set (0.000 sec)

MariaDB [40841063]>
```

Figure 3 Union table creation

Employee Table

```
MariaDB [40841063]> CREATE TABLE Employee (  
-> ssn VARCHAR(20) PRIMARY KEY,  
-> name VARCHAR(100) NOT NULL,  
-> address VARCHAR(255),  
-> phone_number VARCHAR(20),  
-> salary DECIMAL(10,2),  
-> union_id INT NOT NULL,  
-> CONSTRAINT fk_employee_union  
-> FOREIGN KEY (union_id)  
-> REFERENCES Union_Table(union_id)  
-> );  
Query OK, 0 rows affected (0.008 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Employee            |  
| Union_Table         |  
+-----+  
2 rows in set (0.000 sec)  
  
MariaDB [40841063]>
```

Figure 4 Employee Table

Airplane models

```
MariaDB [40841063]> CREATE TABLE Airplane_models (  
    -> model_number VARCHAR(20) PRIMARY KEY,  
    -> capacity INT,  
    -> weight DECIMAL(10,2),  
    -> seats INT,  
    -> max_speed DECIMAL(10,2)  
    -> );  
Query OK, 0 rows affected (0.006 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane_models    |  
| Employee            |  
| Union_Table         |  
+-----+  
3 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 5 Airplane model table

Airplane Table

```
MariaDB [40841063]> CREATE TABLE Airplane (  
    -> registration_number VARCHAR(20) PRIMARY KEY,  
    -> model_number VARCHAR(20) NOT NULL,  
    -> CONSTRAINT fk_ariplane_model  
    -> FOREIGN KEY (model_number)  
    -> REFERENCES Airplane_models(model_number)  
    -> );  
Query OK, 0 rows affected (0.006 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane            |  
| Airplane_models     |  
| Employee            |  
| Union_Table         |  
+-----+  
4 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 6 Airplane Table

Technicians Table

```
MariaDB [40841063]> CREATE TABLE Technicians (  
-> ssn VARCHAR (20) PRIMARY KEY,  
-> hourly_rate DECIMAL(10,2),  
-> specialization_area VARCHAR(20),  
-> CONSTRAINT fk_technician_employee  
-> FOREIGN KEY (ssn)  
-> REFERENCES Employee(ssn));  
Query OK, 0 rows affected (0.021 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane            |  
| Airplane_models     |  
| Employee            |  
| Technicians         |  
| Union_Table         |  
+-----+  
5 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 7 Technician Table

Traffic controller table

```
MariaDB [40841063]> CREATE TABLE Traffic_controllers (  
  -> ssn VARCHAR(20) PRIMARY KEY,  
  -> hourly_rate DECIMAL(10,2),  
  -> last_examination_date DATE,  
  -> CONSTRAINT fk_traffic_controller_employee  
  -> FOREIGN KEY (ssn)  
  -> REFERENCES Employee(ssn));  
Query OK, 0 rows affected (0.008 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane           |  
| Airplane_models    |  
| Employee            |  
| Technicians         |  
| Traffic_controllers |  
| Union_Table         |  
+-----+  
6 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 8 Traffic controller

Technician expertise

```
MariaDB [40841063]> CREATE TABLE Technician_experties (  
-> model_number VARCHAR(20),  
-> ssn VARCHAR(20),  
-> PRIMARY KEY (model_number, ssn),  
-> CONSTRAINT fk_expertise_model  
-> FOREIGN KEY (model_number)  
-> REFERENCES Airplane_models(model_number),  
-> CONSTRAINT fk_expertise_technician  
-> FOREIGN KEY (ssn)  
-> REFERENCES Technicians(ssn));  
Query OK, 0 rows affected (0.006 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane            |  
| Airplane_models     |  
| Employee            |  
| Technician_experties |  
| Technicians         |  
| Traffic_controllers |  
| Union_Table         |  
+-----+  
7 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 9 Technician expertise table

Airworthiness tests model table

```
MariaDB [40841063]> CREATE TABLE Airworthiness_tests_model (  
    -> test_number INT PRIMARY KEY,  
    -> name VARCHAR(20) NOT NULL,  
    -> max_score DECIMAL(5,2));  
Query OK, 0 rows affected (0.005 sec)  
  
MariaDB [40841063]> show tables;  
+-----+  
| Tables_in_40841063 |  
+-----+  
| Airplane            |  
| Airplane_models     |  
| Airworthiness_tests_model |  
| Employee            |  
| Technician_experties |  
| Technicians          |  
| Traffic_controllers  |  
| Union_Table          |  
+-----+  
8 rows in set (0.000 sec)  
  
MariaDB [40841063]> |
```

Figure 10 Airworthiness test model table

Test events table

```
MariaDB [40841063]> CREATE TABLE Test_events (  
  -> test_event_number INT AUTO_INCREMENT PRIMARY KEY,  
  -> test_number INT NOT NULL,  
  -> registration_number VARCHAR(20) NOT NULL,  
  -> ssn VARCHAR(20) NOT NULL,  
  -> test_date DATE,  
  -> hours_spent DECIMAL(5,2),  
  -> score DECIMAL(5,2),  
  -> CONSTRAINT fk_test_event_test  
  -> FOREIGN KEY(test_number)  
  -> REFERENCES Airworthiness_tests_model(test_number),  
  -> CONSTRAINT fk_test_event_airplane  
  -> FOREIGN KEY(registration_number)  
  -> REFERENCES Airplane(registration_number),  
  -> CONSTRAINT fk_test_event_technician  
  -> FOREIGN KEY(ssn)  
  -> REFERENCES Technicians(ssn));
```

Query OK, 0 rows affected (0.008 sec)

```
MariaDB [40841063]> show tables;
```

Tables_in_40841063
Airplane
Airplane_models
Airworthiness_tests_model
Employee
Technician_experties
Technicians
Test_events
Traffic_controllers
Union_Table

9 rows in set (0.000 sec)

```
MariaDB [40841063]> |
```

Figure 11 Test events table

Data Manipulation Language (DML) Implementation

This section includes data insertion into the created tables. Each table will have at least 15 records to ensure that the queries will give better insights and more realistic information. The statements are captured from notepad.

Union Table Data Insertion

```
INSERT INTO Union_Table (union_name, membership_status) VALUES  
( 'Technician Union', 'Active'),  
( 'Controller Union', 'Active'),  
( 'Maintenance Union', 'Active'),  
( 'Staff Union', 'Active'),  
( 'Workers Union', 'Active');
```

Figure 12 Union Table Data insertion

Employee Table Data Insertion

```
INSERT INTO Employee
(ssn, name, address, phone_number, salary, union_id) VALUES
('EMP001', 'Aung Min', '12 Airport Road', '0910000001', 42000.00, 1),
('EMP002', 'Ko Zaw', '15 Aviation Street', '0910000002', 43000.00, 1),
('EMP003', 'Htet Lin', '18 Terminal Road', '0910000003', 44000.00, 3),
('EMP004', 'Min Thu', '21 Hangar Avenue', '0910000004', 45000.00, 3),
('EMP005', 'Kyaw Zin', '24 Airport Road', '0910000005', 46000.00, 1),
('EMP006', 'Sai Tun', '27 Aviation Street', '0910000006', 47000.00, 3),
('EMP007', 'Thura Aung', '30 Terminal Road', '0910000007', 48000.00, 1),
('EMP008', 'Nay Lin', '33 Hangar Avenue', '0910000008', 49000.00, 3),
('EMP009', 'Myo Min', '36 Airport Road', '0910000009', 50000.00, 1),
('EMP010', 'Aung Ko', '39 Aviation Street', '0910000010', 51000.00, 3),
('EMP011', 'Zaw Min', '42 Terminal Road', '0910000011', 52000.00, 1),
('EMP012', 'Tun Tun', '45 Hangar Avenue', '0910000012', 53000.00, 3),
('EMP013', 'Ye Min', '48 Airport Road', '0910000013', 54000.00, 1),
('EMP014', 'Lin Htet', '51 Aviation Street', '0910000014', 55000.00, 3),
('EMP015', 'Aung Hla', '54 Terminal Road', '0910000015', 56000.00, 1),

('EMP016', 'Mingalar Win', '57 Airport Road', '0910000016', 57000.00, 2),
('EMP017', 'Thet Naing', '60 Control Tower Road', '0910000017', 58000.00, 2),
('EMP018', 'Kyaw Kyaw', '63 Aviation Street', '0910000018', 59000.00, 2),
('EMP019', 'Htun Htun', '66 Airport Road', '0910000019', 60000.00, 2),
('EMP020', 'Aung Paing', '69 Control Tower Road', '0910000020', 61000.00, 2),
('EMP021', 'Myat Ko', '72 Aviation Street', '0910000021', 62000.00, 2),
('EMP022', 'Thaw Zin', '75 Airport Road', '0910000022', 63000.00, 2),
('EMP023', 'Pyae Phyo', '78 Control Tower Road', '0910000023', 64000.00, 2),
('EMP024', 'Kaung Myat', '81 Aviation Street', '0910000024', 65000.00, 2),
('EMP025', 'Myo Thant', '84 Airport Road', '0910000025', 66000.00, 2),
('EMP026', 'Zin Min', '87 Control Tower Road', '0910000026', 67000.00, 2),
('EMP027', 'Hein Htet', '90 Aviation Street', '0910000027', 68000.00, 2),
('EMP028', 'Sai Aung', '93 Airport Road', '0910000028', 69000.00, 2),
('EMP029', 'Nay Htet', '96 Control Tower Road', '0910000029', 70000.00, 2),
('EMP030', 'Min Zaw', '99 Aviation Street', '0910000030', 71000.00, 2);
```

Figure 13 Employee Table Data Insertion

Airplane Models

```
INSERT INTO Airplane_models
(model_number, capacity, weight, seats, max_speed) VALUES
('A320-100', 150, 73500.00, 150, 828.00),
('A320-200', 180, 77000.00, 180, 828.00),
('A321-200', 220, 89000.00, 220, 833.00),
('A321neo', 240, 97000.00, 240, 840.00),
('A330-200', 250, 242000.00, 250, 871.00),
('A330-300', 300, 242000.00, 300, 871.00),
('A350-900', 325, 280000.00, 325, 903.00),
('A350-1000', 350, 316000.00, 350, 903.00),
('B737-800', 189, 79000.00, 189, 839.00),
('B737-900', 220, 85000.00, 220, 839.00),
('B737-MAX8', 210, 82100.00, 210, 839.00),
('B777-200', 314, 297000.00, 314, 905.00),
('B777-300', 368, 299000.00, 368, 905.00),
('B787-8', 242, 228000.00, 242, 903.00),
('B787-9', 290, 254000.00, 290, 903.00);
```

Figure 14 Airplane Model Data Insertion

Airplanes

```
INSERT INTO Airplane
(registration_number, model_number) VALUES
('XY-001', 'A320-100'),
('XY-002', 'A320-200'),
('XY-003', 'A321-200'),
('XY-004', 'A321neo'),
('XY-005', 'A330-200'),
('XY-006', 'A330-300'),
('XY-007', 'A350-900'),
('XY-008', 'A350-1000'),
('XY-009', 'B737-800'),
('XY-010', 'B737-900'),
('XY-011', 'B737-MAX8'),
('XY-012', 'B777-200'),
('XY-013', 'B777-300'),
('XY-014', 'B787-8'),
('XY-015', 'B787-9');
```

Figure 15 Airplane Data Insertion

Technicians

```
INSERT INTO Technicians
(ssn, hourly_rate, specialization_area) VALUES
('EMP001', 28.00, 'Engine Maintenance'),
('EMP002', 29.00, 'Avionics'),
('EMP003', 30.00, 'Hydraulic Systems'),
('EMP004', 31.00, 'Electrical Systems'),
('EMP005', 32.00, 'Aircraft Structure'),
('EMP006', 33.00, 'Engine Maintenance'),
('EMP007', 34.00, 'Avionics'),
('EMP008', 35.00, 'Landing Gear'),
('EMP009', 36.00, 'Fuel Systems'),
('EMP010', 37.00, 'Electrical Systems'),
('EMP011', 38.00, 'Aircraft Structure'),
('EMP012', 39.00, 'Hydraulic Systems'),
('EMP013', 40.00, 'Avionics'),
('EMP014', 41.00, 'Engine Maintenance'),
('EMP015', 42.00, 'Landing Gear');
```

Figure 16 Technicians Data Insertion

Traffic controllers

```
INSERT INTO Traffic_controllers
(ssn, hourly_rate, last_examination_date) VALUES
('EMP016', 35.00, '2026-01-10'),
('EMP017', 36.00, '2026-01-15'),
('EMP018', 37.00, '2026-01-20'),
('EMP019', 38.00, '2026-01-25'),
('EMP020', 39.00, '2026-02-01'),
('EMP021', 40.00, '2026-02-05'),
('EMP022', 41.00, '2026-02-10'),
('EMP023', 42.00, '2026-02-15'),
('EMP024', 43.00, '2026-02-20'),
('EMP025', 44.00, '2026-02-25'),
('EMP026', 45.00, '2026-03-01'),
('EMP027', 46.00, '2026-03-05'),
('EMP028', 47.00, '2026-03-10'),
('EMP029', 48.00, '2026-03-15'),
('EMP030', 49.00, '2026-03-20');
```

Figure 17 Traffic Controller Data Insertion

Technician expertise

```
INSERT INTO Technician_experties
(model_number, ssn) VALUES
('A320-100', 'EMP001'),
('A320-200', 'EMP002'),
('A321-200', 'EMP003'),
('A321neo', 'EMP004'),
('A330-200', 'EMP005'),
('A330-300', 'EMP006'),
('A350-900', 'EMP007'),
('A350-1000', 'EMP008'),
('B737-800', 'EMP009'),
('B737-900', 'EMP010'),
('B737-MAX8', 'EMP011'),
('B777-200', 'EMP012'),
('B777-300', 'EMP013'),
('B787-8', 'EMP014'),
('B787-9', 'EMP015');
```

Figure 18 Technician expertise Data Insertion

Airworthiness Tests

```
INSERT INTO Airworthiness_tests_model
(test_number, name, max_score) VALUES
(1, 'Engine Performance', 100.00),
(2, 'Avionics System', 100.00),
(3, 'Hydraulic System', 100.00),
(4, 'Electrical System', 100.00),
(5, 'Aircraft Structure', 100.00),
(6, 'Landing Gear', 100.00),
(7, 'Fuel System', 100.00),
(8, 'Flight Control', 100.00),
(9, 'Navigation System', 100.00),
(10, 'Communication', 100.00),
(11, 'Cabin Pressure', 100.00),
(12, 'Emergency Equipment', 100.00),
(13, 'Brake System', 100.00),
(14, 'Safety Equipment', 100.00),
(15, 'Airworthiness Test', 100.00);
```

Figure 19 Airworthiness Tests Data Insertion

Test events

```
MariaDB [40841063]> SELECT * FROM Test_events;
```

test_event_number	test_number	registration_number	ssn	test_date	hours_spent	score
16	1	XY-001	EMP001	2026-04-01	2.50	94.00
17	2	XY-002	EMP002	2026-04-03	2.00	91.00
18	3	XY-003	EMP003	2026-04-05	2.50	88.00
19	4	XY-004	EMP004	2026-04-07	3.00	95.00
20	5	XY-005	EMP005	2026-04-09	4.00	89.00
21	6	XY-006	EMP006	2026-04-11	2.50	93.00
22	7	XY-007	EMP007	2026-04-13	3.00	97.00
23	8	XY-008	EMP008	2026-04-15	3.50	92.00
24	9	XY-009	EMP009	2026-04-17	2.00	96.00
25	10	XY-010	EMP010	2026-04-19	2.50	90.00
26	11	XY-011	EMP011	2026-04-21	3.00	87.00
27	12	XY-012	EMP012	2026-04-23	4.00	94.00
28	13	XY-013	EMP013	2026-04-25	3.50	98.00
29	14	XY-014	EMP014	2026-04-27	2.50	91.00
30	15	XY-015	EMP015	2026-04-29	3.00	95.00

15 rows in set (0.001 sec)

```
MariaDB [40841063]>
```

Figure 20 Test event data insertion

```
MariaDB [40841063]> INSERT INTO Test_events
-> (test_number, registration_number, ssn, test_date, hours_spent, score) VALUES
-> (2, 'XY-001', 'EMP002', '2026-05-01', 3.00, 96.00),
-> (1, 'XY-002', 'EMP001', '2026-05-03', 2.50, 89.00),
-> (4, 'XY-003', 'EMP004', '2026-05-05', 3.00, 91.00),
-> P005',(3, 'XY-004', 'EMP003', '2026-05-07', 2.00, 97.00),
-> (6, 'XY-005', 'EMP006', '2026-05-09', 3.50, 92.00),
-> (5, 'XY-006', 'EMP005', '2026-05-11', 3.00, 90.00),
-> (8, 'XY-007', 'EMP008', '2026-05-13', 2.50, 95.00),
-> (7, 'XY-008', 'EMP007', '2026-05-15', 4.00, 88.00),
-> (10, 'XY-009', 'EMP010', '2026-05-17', 2.50, 94.00),
-> (9, 'XY-010', 'EMP009', '2026-05-19', 3.00, 92.00),
-> (12, 'XY-011', 'EMP012', '2026-05-21', 3.50, 90.00),
-> (11, 'XY-012', 'EMP011', '2026-05-23', 2.50, 96.00),
-> (14, 'XY-013', 'EMP014', '2026-05-25', 3.00, 93.00),
-> (13, 'XY-014', 'EMP013', '2026-05-27', 3.50, 95.00),
-> (15, 'XY-015', 'EMP015', '2026-05-29', 2.00, 98.00);\c
Query OK, 15 rows affected (0.003 sec)
Records: 15 Duplicates: 0 Warnings: 0

MariaDB [40841063]> SELECT * FROM Test_events;
```

test_event_number	test_number	registration_number	ssn	test_date	hours_spent	score
16	1	XY-001	EMP001	2026-04-01	2.50	94.00
17	2	XY-002	EMP002	2026-04-03	2.00	91.00
18	3	XY-003	EMP003	2026-04-05	2.50	88.00
19	4	XY-004	EMP004	2026-04-07	3.00	95.00
20	5	XY-005	EMP005	2026-04-09	4.00	89.00
21	6	XY-006	EMP006	2026-04-11	2.50	93.00
22	7	XY-007	EMP007	2026-04-13	3.00	97.00
23	8	XY-008	EMP008	2026-04-15	3.50	92.00
24	9	XY-009	EMP009	2026-04-17	2.00	96.00
25	10	XY-010	EMP012	2026-04-19	2.50	90.00
26	11	XY-011	EMP011	2026-04-21	3.00	87.00
27	12	XY-012	EMP012	2026-04-23	4.00	94.00
28	13	XY-013	EMP013	2026-04-25	3.50	98.00
29	14	XY-014	EMP001	2026-04-27	2.50	91.00
30	15	XY-015	EMP012	2026-04-29	3.00	95.00
31	2	XY-001	EMP002	2026-05-01	3.00	96.00
32	1	XY-002	EMP001	2026-05-03	2.50	89.00
33	4	XY-003	EMP004	2026-05-05	3.00	91.00
34	3	XY-004	EMP003	2026-05-07	2.00	97.00
35	6	XY-005	EMP006	2026-05-09	3.50	92.00
36	5	XY-006	EMP005	2026-05-11	3.00	90.00
37	8	XY-007	EMP008	2026-05-13	2.50	95.00
38	7	XY-008	EMP007	2026-05-15	4.00	88.00
39	10	XY-009	EMP010	2026-05-17	2.50	94.00
40	9	XY-010	EMP009	2026-05-19	3.00	92.00
41	12	XY-011	EMP012	2026-05-21	3.50	90.00
42	11	XY-012	EMP011	2026-05-23	2.50	96.00
43	14	XY-013	EMP014	2026-05-25	3.00	93.00

Figure 21 Inserting more rows in Test events for screenshot

Task E – Queries

This section of the report focuses on querying the data in the table for more insight. Below five query results will be shown as screenshots and the insights they provide.

Query 1 – Aircraft Test Performance Summary

```
Database changed
MariaDB [40841063]> SELECT am.model_number, a.registration_number, t.ssn AS technician_ssn
-> , COUNT(te.test_event_number) AS total_test_events, SUM(te.hours_spent) AS total_test_hours,
-> AVG(te.score) AS average_score
-> FROM Test_events te
-> JOIN Airplane a ON te.registration_number = a.registration_number
-> JOIN Airplane_models am ON a.model_number = am.model_number
-> JOIN Technicians t ON te.ssn = t.ssn
-> GROUP BY am.model_number;
```

model_number	registration_number	technician_ssn	total_test_events	total_test_hours	average_score
A320-100	XY-001	EMP001	2	5.50	95.000000
A320-200	XY-002	EMP002	2	4.50	90.000000
A321-200	XY-003	EMP003	2	5.50	89.500000
A321neo	XY-004	EMP004	2	5.00	96.000000
A330-200	XY-005	EMP005	2	7.50	90.500000
A330-300	XY-006	EMP006	2	5.50	91.500000
A350-1000	XY-008	EMP008	2	7.50	90.000000
A350-900	XY-007	EMP007	2	5.50	96.000000
B737-800	XY-009	EMP009	2	4.50	95.000000
B737-900	XY-010	EMP012	2	5.50	91.000000
B737-MAX8	XY-011	EMP011	2	6.50	88.500000
B777-200	XY-012	EMP012	2	6.50	95.000000
B777-300	XY-013	EMP013	2	6.50	95.500000
B787-8	XY-014	EMP001	2	6.00	93.000000
B787-9	XY-015	EMP012	2	5.00	96.500000

```
15 rows in set (0.001 sec)

MariaDB [40841063]>
```

Figure 22 Airplane Test Performance Summary

This query takes the average of testing metrics for each airplane model number. Links to the test events table and also airplane model details and the technician's records to determine the overall number of test events, total hours elapsed and average score per model. This gives an in-depth view of the degree of testing and evaluation of every aircraft model.

Query 2 – Employee Annual Performance

```
MariaDB [40841063]> SELECT e.ssn AS technician_ssn, e.name AS technician_name,  
-> YEAR(te.test_date) AS test_year, COUNT(te.test_event_number) AS total_tests, SUM(te.hours_spent)  
-> FROM Employee e  
-> JOIN Technicians t ON e.ssn = t.ssn  
-> JOIN Test_events te ON t.ssn = te.ssn  
-> GROUP BY e.ssn, e.name, YEAR(te.test_date)  
-> ORDER BY total_tests DESC;
```

technician_ssn	technician_name	test_year	total_tests	SUM(te.hours_spent)
EMP012	Tun Tun	2026	4	13.00
EMP001	Aung Min	2026	3	7.50
EMP004	Min Thu	2026	2	6.00
EMP005	Kyaw Zin	2026	2	7.00
EMP006	Sai Tun	2026	2	6.00
EMP007	Thura Aung	2026	2	7.00
EMP008	Nay Lin	2026	2	6.00
EMP009	Myo Min	2026	2	5.00
EMP011	Zaw Min	2026	2	5.50
EMP013	Ye Min	2026	2	7.00
EMP002	Ko Zaw	2026	2	5.00
EMP003	Htet Lin	2026	2	4.50
EMP010	Aung Ko	2026	1	2.50
EMP014	Lin Htet	2026	1	3.00
EMP015	Aung Hla	2026	1	2.00

15 rows in set (0.002 sec)

```
MariaDB [40841063]>
```

Figure 23 Employee Annual Performance Query

This query returns productivity which is the number of test events performed by the technician in a calendar year. It pulls the year from the test date and also merges the employee and technician table to obtain employee name matches with the corresponding ID numbers. The results can be sorted in descending order so you can see at a glance which technicians are doing better.

Query 3 – Recently Examined Traffic Controllers

```
MariaDB [40841063]> SELECT
  ->     e.ssn,
  ->     e.name AS controller_name,
  ->     tc.last_examination_date
  -> FROM Employee e
  -> JOIN Traffic_controllers tc ON e.ssn = tc.ssn
  -> ORDER BY tc.last_examination_date DESC;
+-----+-----+-----+
| ssn    | controller_name | last_examination_date |
+-----+-----+-----+
| EMP030 | Min Zaw        | 2026-03-20            |
| EMP029 | Nay Htet       | 2026-03-15            |
| EMP028 | Sai Aung       | 2026-03-10            |
| EMP027 | Hein Htet      | 2026-03-05            |
| EMP026 | Zin Min        | 2026-03-01            |
| EMP025 | Myo Thant      | 2026-02-25            |
| EMP024 | Kaung Myat     | 2026-02-20            |
| EMP023 | Pyae Phyo      | 2026-02-15            |
| EMP022 | Thaw Zin       | 2026-02-10            |
| EMP021 | Myat Ko        | 2026-02-05            |
| EMP020 | Aung Paing     | 2026-02-01            |
| EMP019 | Htun Htun      | 2026-01-25            |
| EMP018 | Kyaw Kyaw      | 2026-01-20            |
| EMP017 | Thet Naing     | 2026-01-15            |
| EMP016 | Mingalar Win   | 2026-01-10            |
+-----+-----+-----+
15 rows in set (0.000 sec)

MariaDB [40841063]>
```

Figure 24 Recently examined traffic controllers

This query is used to obtain personal data from traffic controllers, and the date of their last medical or professional examination. This will merge this table with the core employee table and return a cleaned sorted list with the most recent examination date at the top. This enables administrators to easily keep track of compliance and determine when the controllers are due for renewal.

Query 4 – Aircraft safety check

```

SELECT
->    am.model_number,
->    a.registration_number,
->    e.name AS technician_name,
->    te.score AS latest_score,
->    hist_avg.avg_historical_score AS historical_average,
->    CASE
->        WHEN te.score >= hist_avg.avg_historical_score THEN 'Pass'
->        ELSE 'Review Required'
->    END AS safety_status
-> FROM Test_events te
-> JOIN Airplane a ON te.registration_number = a.registration_number
-> JOIN Airplane_models am ON a.model_number = am.model_number
-> JOIN Technicians t ON te.ssn = t.ssn
-> JOIN Employee e ON t.ssn = e.ssn
-> JOIN (
->     SELECT registration_number, AVG(score) AS avg_historical_score
->     FROM Test_events
->     GROUP BY registration_number
-> ) hist_avg ON te.registration_number = hist_avg.registration_number
-> WHERE te.test_date = (
->     SELECT MAX(test_date)
->     FROM Test_events
->     WHERE registration_number = te.registration_number
-> );

```

Figure 25 Airplane safety check query

model_number	registration_number	technician_name	latest_score	historical_average	safety_status
A320-100	XY-001	Ko Zaw	96.00	95.000000	Pass
A320-200	XY-002	Aung Min	89.00	90.000000	Review Required
A321-200	XY-003	Min Thu	91.00	89.500000	Pass
A321neo	XY-004	Htet Lin	97.00	96.000000	Pass
A330-200	XY-005	Sai Tun	92.00	90.500000	Pass
A330-300	XY-006	Kyaw Zin	90.00	91.500000	Review Required
A350-1000	XY-008	Thura Aung	88.00	90.000000	Review Required
A350-900	XY-007	Nay Lin	95.00	96.000000	Review Required
B737-800	XY-009	Aung Ko	94.00	95.000000	Review Required
B737-900	XY-010	Myo Min	92.00	91.000000	Pass
B737-MAX8	XY-011	Tun Tun	90.00	88.500000	Pass
B777-200	XY-012	Zaw Min	96.00	95.000000	Pass
B777-300	XY-013	Lin Htet	93.00	95.500000	Review Required
B787-8	XY-014	Ye Min	95.00	93.000000	Pass
B787-9	XY-015	Aung Hla	98.00	96.500000	Pass

Figure 26 Aircraft safety check results

This query will compare the latest test score of an aircraft to the average test score for all previous tests conducted by that aircraft. This uses one set of nested subqueries to select the most recent date of the test for each registration number, and another to obtain the baseline average score. Conditional logic then automatically applies a "Pass" rating to the aircraft if the score remains within the historical ratings, or a "Review Required" rating if the score falls outside of the historical ratings.

Query 5 – Technician Expertise summary

```
MariaDB [40841063]> SELECT
->     e.ssn,
->     e.name AS technician_name,
->     am.model_number,
->     t.specialization_area
-> FROM Employee e
-> JOIN Technicians t ON e.ssn = t.ssn
-> JOIN Technician_experties te ON t.ssn = te.ssn
-> JOIN Airplane_models am ON te.model_number = am.model_number
-> ORDER BY am.model_number, e.name;
```

ssn	technician_name	model_number	specialization_area
EMP001	Aung Min	A320-100	Engine Maintenance
EMP002	Ko Zaw	A320-200	Avionics
EMP003	Htet Lin	A321-200	Hydraulic Systems
EMP004	Min Thu	A321neo	Electrical Systems
EMP005	Kyaw Zin	A330-200	Aircraft Structure
EMP006	Sai Tun	A330-300	Engine Maintenance
EMP008	Nay Lin	A350-1000	Landing Gear
EMP007	Thura Aung	A350-900	Avionics
EMP009	Myo Min	B737-800	Fuel Systems
EMP010	Aung Ko	B737-900	Electrical Systems
EMP011	Zaw Min	B737-MAX8	Aircraft Structure
EMP012	Tun Tun	B777-200	Hydraulic Systems
EMP013	Ye Min	B777-300	Avionics
EMP014	Lin Htet	B787-8	Engine Maintenance
EMP015	Aung Hla	B787-9	Landing Gear

```
15 rows in set (0.001 sec)

MariaDB [40841063]>
```

Figure 27 Technician Experties Summary

This query relates to a specific type of aircraft that they can work on and also a specific type of expertise, like engine or avionics. Linking employee records via technician expertise tables to produce a comprehensive list of employees' technical skills. By listing them in order of model number and technician name it is much easier to assign an appropriate technician to a maintenance issue.

Presentation of Query Results

This section will include the presentation of query results compared to the test plan in part 1. Five queries will be compared to their test plan with sample data and discussed whether they achieve the expected results.

Query 1

Model Number	Registration Number	Technician SSN	Number of Tests	Total Test Hours	Average Test Score
A320	101	1001	3	7.5	88.33
A320	102	1002	2	7.0	75.00
B737	103	1003	2	4.5	90.00

Table 8 Query 1 Test Plan

model_number	registration_number	technician_ssn	total_test_events	total_test_hours	average_score
A320-100	XY-001	EMP001	2	5.50	95.000000
A320-200	XY-002	EMP002	2	4.50	90.000000
A321-200	XY-003	EMP003	2	5.50	89.500000
A321neo	XY-004	EMP004	2	5.00	96.000000
A330-200	XY-005	EMP005	2	7.50	90.500000
A330-300	XY-006	EMP006	2	5.50	91.500000
A350-1000	XY-008	EMP008	2	7.50	90.000000
A350-900	XY-007	EMP007	2	5.50	96.000000
B737-800	XY-009	EMP009	2	4.50	95.000000
B737-900	XY-010	EMP012	2	5.50	91.000000
B737-MAX8	XY-011	EMP011	2	6.50	88.500000
B777-200	XY-012	EMP012	2	6.50	95.000000
B777-300	XY-013	EMP013	2	6.50	95.500000
B787-8	XY-014	EMP001	2	6.00	93.000000
B787-9	XY-015	EMP012	2	5.00	96.500000

Table 9 Query 1 Actual Results

Query 2

SSN	Technician Name	Test Year	Total Tests	Total Hours
1001	John Smith	2026	5	12.5
1002	Sarah Brown	2026	2	7.0

Table 10 Query 2 Test Plan

technician_ssn	technician_name	test_year	total_tests	SUM(te.hours_spent)
EMP012	Tun Tun	2026	4	13.00
EMP001	Aung Min	2026	3	7.50
EMP004	Min Thu	2026	2	6.00
EMP005	Kyaw Zin	2026	2	7.00
EMP006	Sai Tun	2026	2	6.00
EMP007	Thura Aung	2026	2	7.00
EMP008	Nay Lin	2026	2	6.00
EMP009	Myo Min	2026	2	5.00
EMP011	Zaw Min	2026	2	5.50
EMP013	Ye Min	2026	2	7.00
EMP002	Ko Zaw	2026	2	5.00
EMP003	Htet Lin	2026	2	4.50
EMP010	Aung Ko	2026	1	2.50
EMP014	Lin Htet	2026	1	3.00
EMP015	Aung Hla	2026	1	2.00

Table 11 Query 2 Actual Results

Query 3

SSN	Name	Last Examination Date
1004	Emily Davis	2026-06-15

Table 12 Query 3 Test Plan

ssn	controller_name	last_examination_date
EMP030	Min Zaw	2026-03-20
EMP029	Nay Htet	2026-03-15
EMP028	Sai Aung	2026-03-10
EMP027	Hein Htet	2026-03-05
EMP026	Zin Min	2026-03-01
EMP025	Myo Thant	2026-02-25
EMP024	Kaung Myat	2026-02-20
EMP023	Pyae Phyo	2026-02-15
EMP022	Thaw Zin	2026-02-10
EMP021	Myat Ko	2026-02-05
EMP020	Aung Paing	2026-02-01
EMP019	Htun Htun	2026-01-25
EMP018	Kyaw Kyaw	2026-01-20
EMP017	Thet Naing	2026-01-15
EMP016	Mingalar Win	2026-01-10

Table 13 Query 3 Actual Results

Query 4

Model Number	Registration Number	Technician Name	Latest Score	Model Average Score	Safety Status
A320	101	John Smith	95	83.00	Pass
A320	102	Sarah Brown	80	83.00	Review Required

Table 14 Query 4 Test Plan

model_number	registration_number	technician_name	latest_score	historical_average	safety_status
A320-100	XY-001	Ko Zaw	96.00	95.000000	Pass
A320-200	XY-002	Aung Min	89.00	90.000000	Review Required
A321-200	XY-003	Min Thu	91.00	89.500000	Pass
A321neo	XY-004	Htet Lin	97.00	96.000000	Pass
A330-200	XY-005	Sai Tun	92.00	90.500000	Pass
A330-300	XY-006	Kyaw Zin	90.00	91.500000	Review Required
A350-1000	XY-008	Thura Aung	88.00	90.000000	Review Required
A350-900	XY-007	Nay Lin	95.00	96.000000	Review Required
B737-800	XY-009	Aung Ko	94.00	95.000000	Review Required
B737-900	XY-010	Myo Min	92.00	91.000000	Pass
B737-MAX8	XY-011	Tun Tun	90.00	88.500000	Pass
B777-200	XY-012	Zaw Min	96.00	95.000000	Pass
B777-300	XY-013	Lin Htet	93.00	95.500000	Review Required
B787-8	XY-014	Ye Min	95.00	93.000000	Pass
B787-9	XY-015	Aung Hla	98.00	96.500000	Pass

Table 15 Query 4 Actual Results

Query 5

SSN	Technician Name	Model Number	Specialization Area
1001	John Smith	A320	Maintenance
1001	John Smith	A330	Technical
1002	Sarah Brown	A320	Storage

Table 16 Query 5 Test Plan

ssn	technician_name	model_number	specialization_area
EMP001	Aung Min	A320-100	Engine Maintenance
EMP002	Ko Zaw	A320-200	Avionics
EMP003	Htet Lin	A321-200	Hydraulic Systems
EMP004	Min Thu	A321neo	Electrical Systems
EMP005	Kyaw Zin	A330-200	Aircraft Structure
EMP006	Sai Tun	A330-300	Engine Maintenance
EMP008	Nay Lin	A350-1000	Landing Gear
EMP007	Thura Aung	A350-900	Avionics
EMP009	Myo Min	B737-800	Fuel Systems
EMP010	Aung Ko	B737-900	Electrical Systems
EMP011	Zaw Min	B737-MAX8	Aircraft Structure
EMP012	Tun Tun	B777-200	Hydraulic Systems
EMP013	Ye Min	B777-300	Avionics
EMP014	Lin Htet	B787-8	Engine Maintenance
EMP015	Aung Hla	B787-9	Landing Gear

Table 17 Query 5 Actual Results

Constraint Testing

This section of the test will test constraints using sample tables. Only some table constraints will be tested since this could increase the report pages. The test will include length constraints and reference constraints.

Name Length Test

```
MariaDB [40841063]> INSERT INTO Airworthiness_tests_model  
-> (test_number, name, max_score) VALUES  
-> (1, 'Engine Performance Test', 100.00);  
ERROR 1406 (22001): Data too long for column 'name' at row 1  
MariaDB [40841063]>
```

Figure 28 Name Length Constraints

```
MariaDB [40841063]> INSERT INTO Union_Table (union_name, membership_status) VALUES  
-> ('Traffic Controller Union', 'Active');  
ERROR 1406 (22001): Data too long for column 'union_name' at row 1  
MariaDB [40841063]>
```

Figure 29 Union name length

These tests check the column character limits by trying to insert strings that are longer than those that have been defined: Inserting 'Airworthiness_tests_model' into the Airworthiness_tests_model column and 'Union_Table' into the Union_Table column. In both cases, the data length constraints are enforced in MariaDB, with ERROR 1406 (Data too long for column). This shows that strict schema validation is on throughout the entities, which means truncating the strings is avoided.

Referential Constraints

```
MariaDB [40841063]> INSERT INTO Employee (ssn, name, address, phone_number, salary, union_id)
VALUES
  -> ('EMP031', 'Kyaw Kyaw', '99 Aviation Street', '09344343432', 70000.00, 7);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`40841063`.`Employee`, CONSTRAINT `fk_employee_union` FOREIGN KEY (`union_id`) REFERENCES `Union_Table` (`union_id`))
MariaDB [40841063]>
```

Figure 30 Employee and Union Table relation testing

This test checks referential integrity by trying to insert a record into an EMP table (EMP031) with an employee_id set to 7, which is not in the referenced Union_Table. The foreign key constraint (fk_employee_union) is enforced successfully by MariaDB rejecting the insert and returning ERROR 1452. This means that orphaned records are not allowed to be created if there is no parent record.

Constraint Summary

By looking at these test results, it is evidence that relation and constraints between the tables are working accordingly. Other constraints such as Primary Key not null values and other constraints will be tested in the demonstration recording. To test primary keys the tester will try to insert duplicated key value, and the database will throw an error. For not null attribute values the test will include 'Null' for the value and the database will throw an error.

Additional Feature – Trigger

A trigger was added to the Test_events table, that logs any changes to test event information automatically. The trigger will store the name of the table, the record id, the type of action taken, the old value, the new value, the date of the update and the username of the user who changed the record in the Update_Log table when updating a record. This will give an audit trail for tracking changes and enhance accountability and reliability of the database.

```
CREATE TABLE Update_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    table_name VARCHAR(100) NOT NULL,  
    record_id VARCHAR(100) NOT NULL,  
    action VARCHAR(20) NOT NULL,  
    old_value VARCHAR(255),  
    new_value VARCHAR(255),  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    changed_by VARCHAR(100) NOT NULL  
);
```

Figure 31 Test events trigger table

The table is not included in the ERD because it has no relation or attributes since its automatically updating table.

```

DELIMITER $$

CREATE TRIGGER trg_test_events_update
AFTER UPDATE ON Test_events
FOR EACH ROW
BEGIN
    INSERT INTO Update_Log
    (
        table_name,
        record_id,
        action,
        old_value,
        new_value,
        changed_at,
        changed_by
    )
    VALUES
    (
        'Test_events',
        OLD.test_event_number,
        'UPDATE',
        CONCAT(
            'test_date=', OLD.test_date,
            ', hours_spent=', OLD.hours_spent,
            ', score=', OLD.score
        ),
        CONCAT(
            'test_date=', NEW.test_date,
            ', hours_spent=', NEW.hours_spent,
            ', score=', NEW.score
        ),
        CURRENT_TIMESTAMP,
        CURRENT_USER()
    );
END$$

DELIMITER ;

```

Figure 32 Test event trigger

The trigger compares OLD and NEW values before and after an update is made to the database, so that it can maintain a clean history of updates. The CURRENT_USER() function is also used to identify the database account that is used to modify the database. The logged information can be viewed in the Update_Log table to help monitor and audit database changes.